STATE-SPACE SEARCH

Motivation and Background

Uninformed Search, Informed Search, Any-Path Search, Optimal Search

n.h.reyes@massey.ac.nz

Reference: **MIT OpenCourseWare**

SEARCH

Background and Motivation

General Idea: Search allows exploring alternatives.

These algorithms provide the conceptual backbone of almost every approach to systematic exploration of alternatives.

Topics for Discussion:

- Background
- Uninformed vs. Informed Searches
- Any Path vs. Optimal Path
- Implementation and Performance

SEARCH

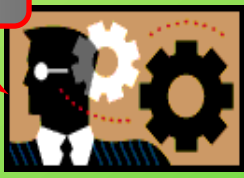
Topics for Discussion

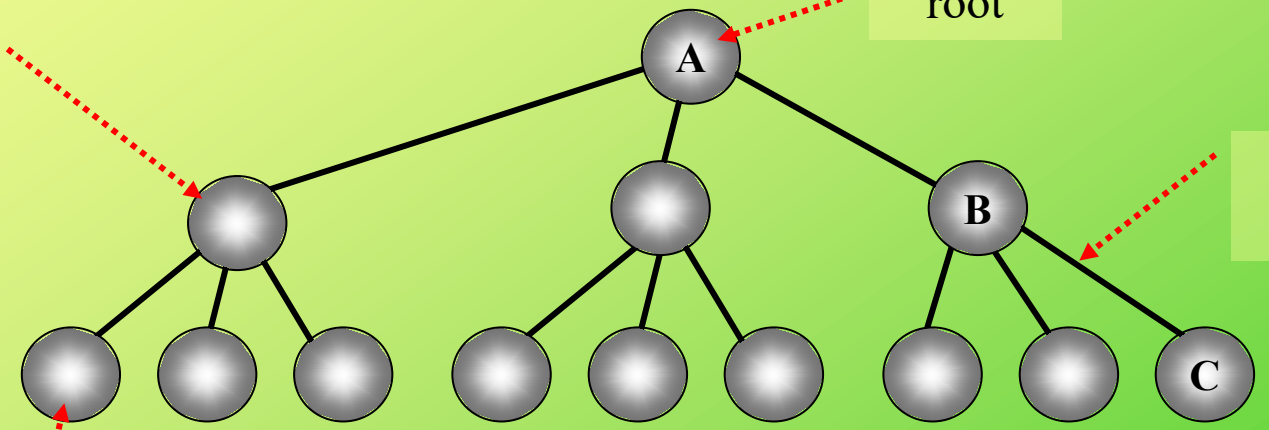
- Basic principles and terminologies
- Graphs vs. Trees
- How to convert a graph into a tree
- Requirements for applying search
- Any-Path Search (Uninformed): Depth-First Search
- Any-Path Search (Uninformed): Breadth-First Search
- Any-Path Search (Informed): Best-First Search
- Hand-simulations

Background and Motivation

Trees and Graphs

No loops are allowed!

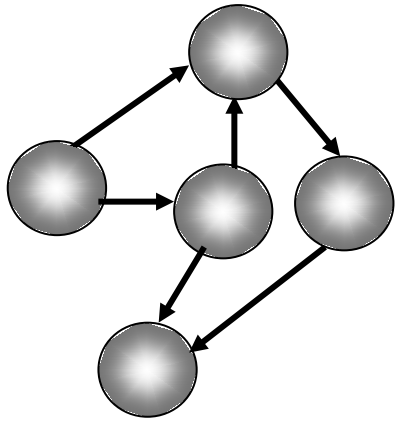




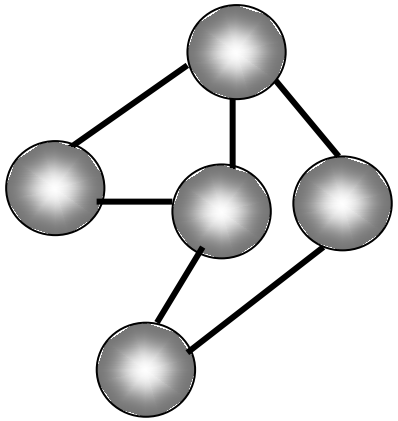
B is parent of C
C is a child of B
A is an ancestor of C
C is descendant of A

Background and Motivation

Directed Graphs
(one-way streets)



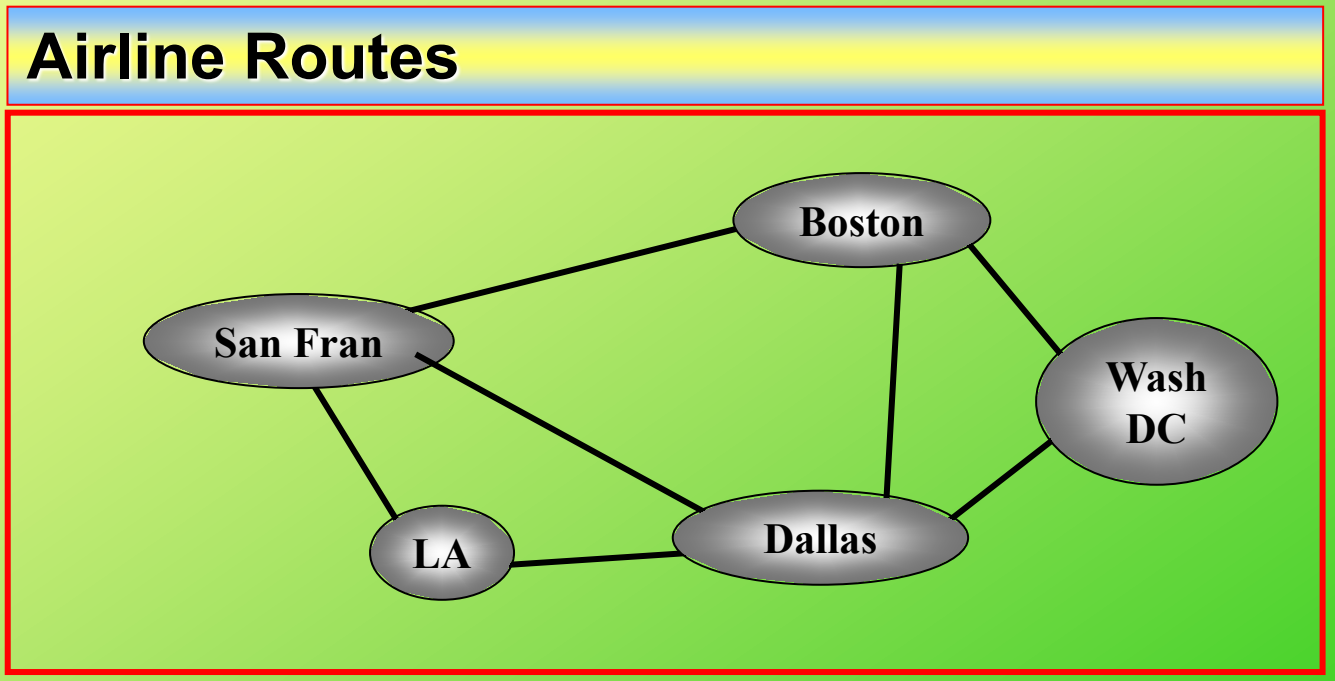
Undirected Graphs
(two-way streets)



Loops are allowed, node can have **multiple parents**

Examples of Graphs

Graphs are everywhere…



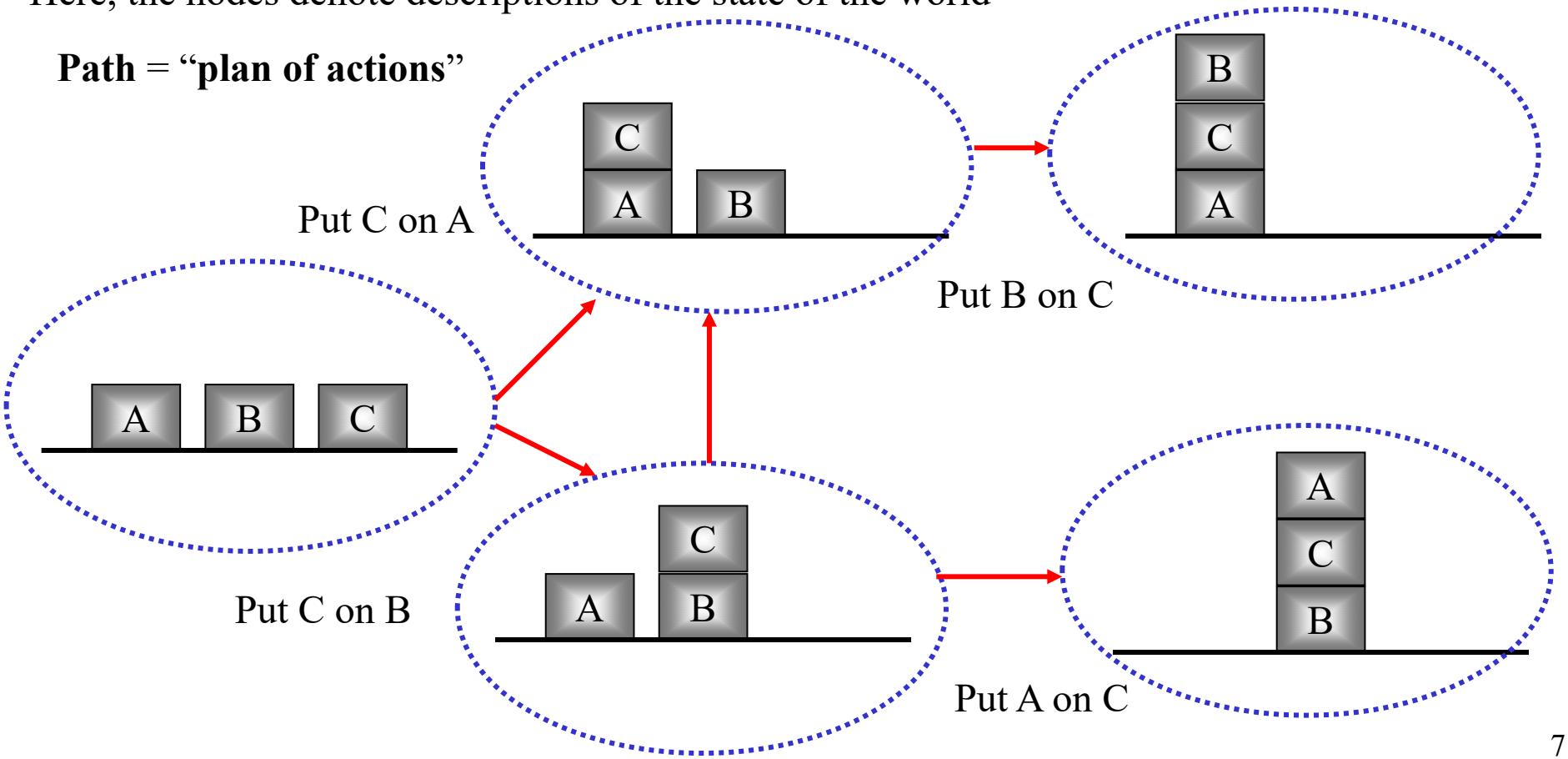
We could be interested in least cost paths, or paths with minimum hops

Examples of Graphs

Planning actions (graph of possible states of the world)

Here, the nodes denote descriptions of the state of the world

Path = "**plan of actions**"



Problem Solving Paradigm

1. **What are the states? (All relevant aspects of the problem)**

- arrangement of parts (to plan an assembly)
- Positions of trucks (to plan package distribution)
- City (to plan a trip)
- set of facts (to prove geometry theorem)

2. **What are the actions? (operators – Deterministic and Discrete)**

- assemble two parts
- move a truck to a new position
- fly to a new city
- apply a theorem to derive a new fact

Problem Solving Paradigm

3. What is the goal test? (Conditions for success)

- all parts in place (to plan an assembly)
- all packages delivered (to plan package distribution)
- reached final destination city (to plan a trip)
- derived goal fact (e.g. to prove geometry theorem)

Graph Search as Tree Search

- **Trees are directed graphs without cycles, and with nodes having <= 1 parent**

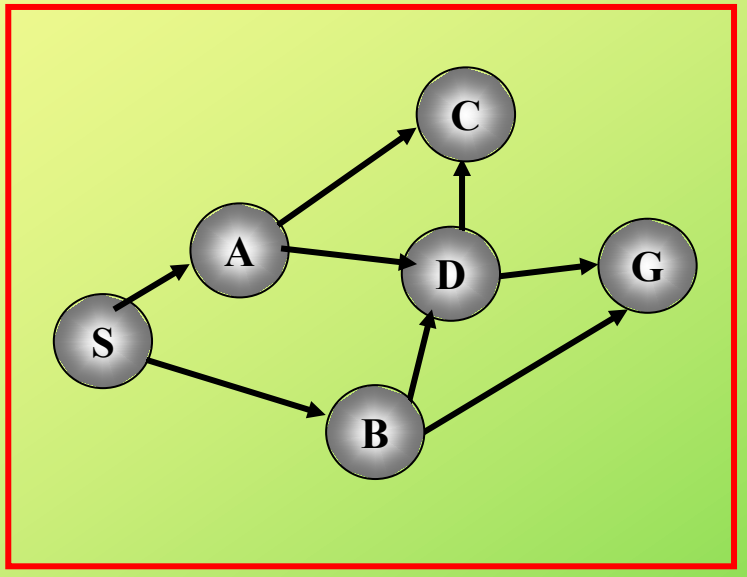
- **We can turn graph search problems into tree search problems by:**

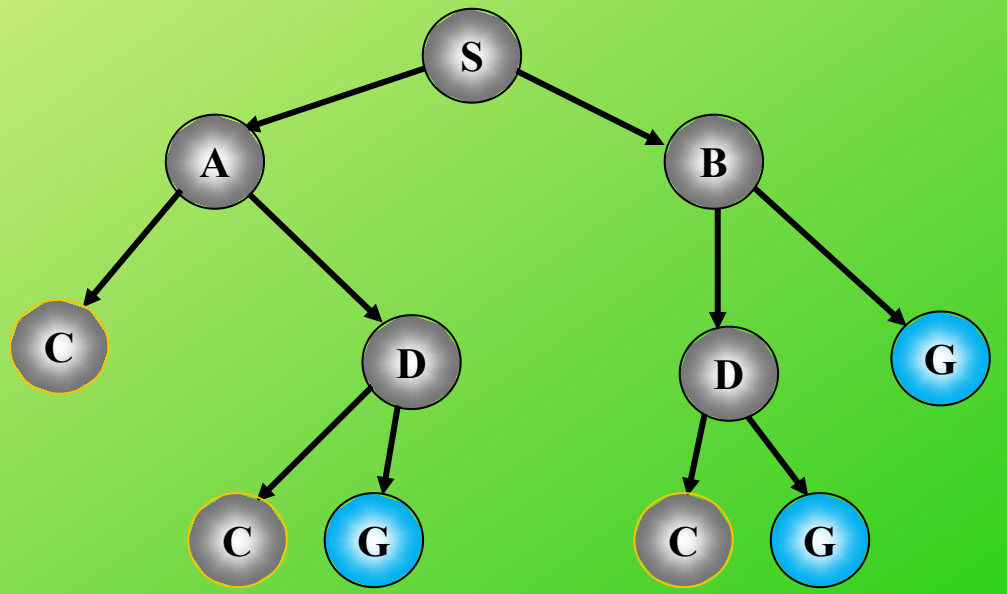
1. Replacing undirected links by 2 directed links
2. Avoiding loops in path (or better yet, keeping track of visited nodes globally)

Graph Search as Tree Search

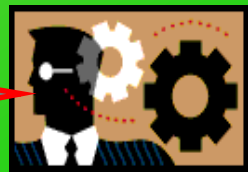
- We can turn **graph search problems** (from S to G) into **tree search problems** by:

1. Replacing undirected links by 2 directed links
2. Avoiding loops in path (or keeping track of visited nodes globally)





Note that nodes may still get duplicated.



SEARCH

Terminologies

State – used to refer to the vertices of the underlying graph that is being searched; that is, states in the problem domain (e.g. a city, an arrangement of blocks or the arrangement of parts in a puzzle)

Search Node – refers to the vertices of the search tree which is being generated by the search algorithm. Each node refers to a state of the world; Many nodes may refer to the same state. Importantly, a node implicitly represents a path (from the start state of the search to the state associated with the node). Because search nodes are part of a search tree, they have a unique ancestor node (except for the root node).

3

STATE-SPACE SEARCH

Any-Path Search

Depth-First Search, Breadth-First Search , Best-First Search

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	

Class	Name	Operation
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	

Class	Name	Operation
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)

Class	Name	Operation
Optimal Uninformed	**Uniform-Cost**	Uses path-length measure. Finds "shortest" path.

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)
Optimal Uninformed	**Uniform-Cost**	Uses path-length measure. Finds "shortest" path.
Optimal Informed	**A***	Uses path "length" measure and heuristic. Finds "shortest" path.

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) ...)

Let S be the start state.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Guarantees that each state is reached only once.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

Implementing the Search Strategies

Depth-First

1. Pick **first** element of Q.

2. Add path extensions to **front** of Q.

Implementing the Search Strategies

Depth-First

1. Pick **first** element of Q.

2. Add path extensions to **front** of Q.

Breadth-First

1. Pick **first** element of Q.

2. Add path extensions to **end** of Q.

SEARCH

Implementing the Search Strategies

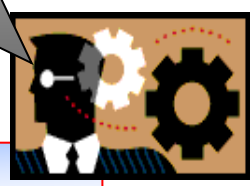
Depth-First (backtracking search)

1. Pick **first** element of Q.
2. Add path extensions to **front** of Q.

Breadth-First

1. Pick **first** element of Q.
2. Add path extensions to **end** of Q.

Heuristic functions are applied in the hope of completing the **search quicker** or **finding a relatively good goal state**.
It does not guarantee finding the "best" path though.



Best-First (greedy search)

1. Pick **best** (measured heuristic value of state) element of Q.
2. Add path extensions **anywhere** in Q (it may be efficient to keep the Q ordered in some way so as to make it easier to find the "best" element.

For our convention in class, add the path extensions to the **front** of the Queue.

Any-Path Algorithms

Simulations of the Algorithms

Let's look at some simulation (hand-computations) of the algorithms first before we characterize them more fully.

159.302 STATE-SPACE SEARCH

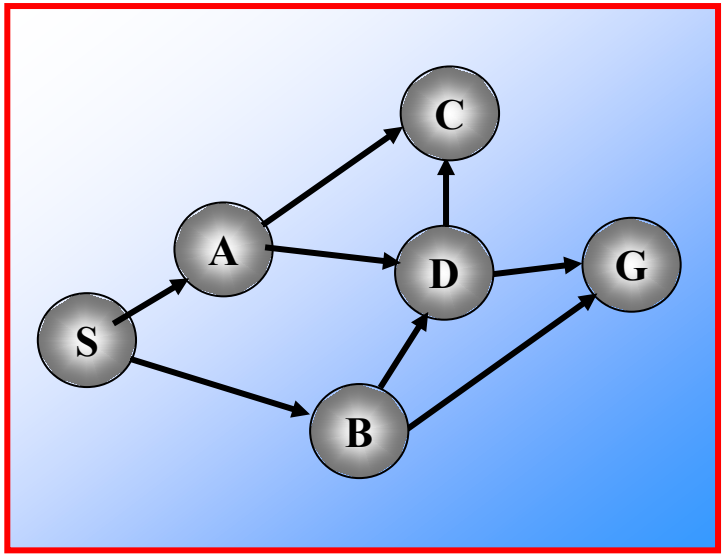
Any-Path Search Examples

Depth-First, Breadth-First, Best-First Search Algorithms

Simulations by hand

Using the following graph, simulate:

1. Depth-First with Visited List
2. Depth-First without Visited List
3. Breadth-First with Visited List
4. Breadth-First without Visited List
5. Best-First with Visited List



Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Guarantees that each state is reached only once.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

Critical Decisions:

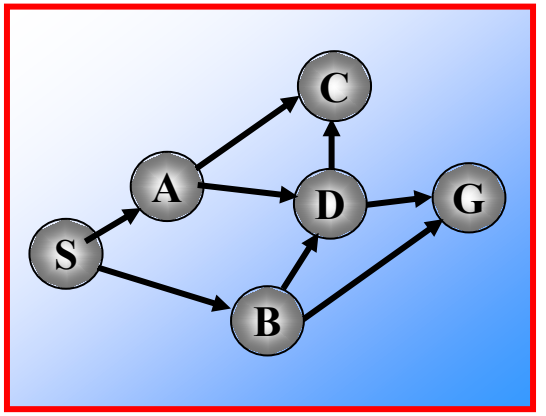
Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

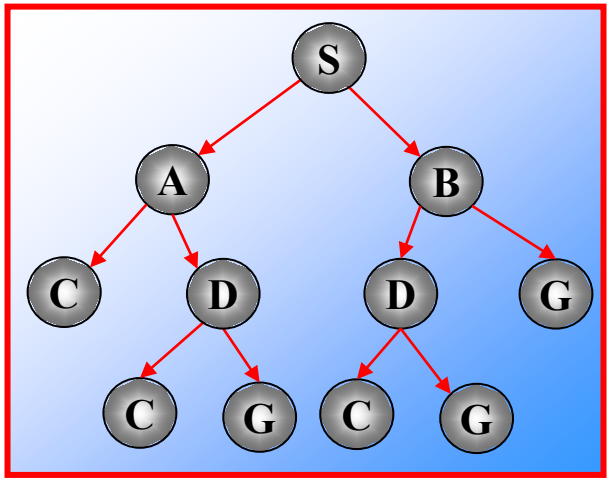
Depth-First with Visited List

Given the following graph, find a path from **S** to **G** using **Depth-First Search with the Visited List.**

Graph



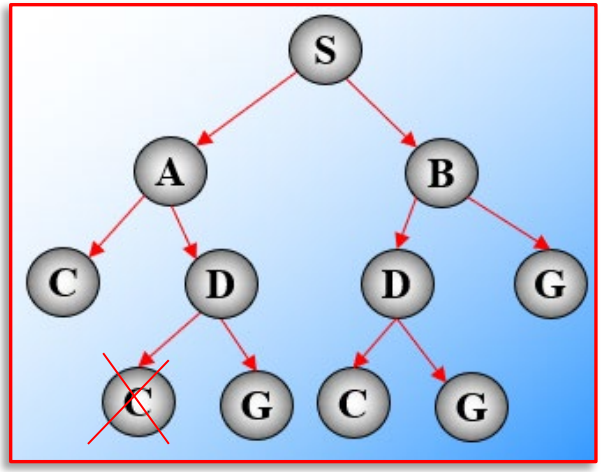
Search tree equivalent



Depth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS)(BS)	S,A,B
2	(AS)	(CAS)(DAS)(BS)	S,A,B,C,D
3	(CAS)	(DAS)(BS)	S,A,B,C,D
4	(DAS)	(GDAS)(BS)	S,A,B,C,D,G
5	(GDAS)	(BS)	

Search Tree



Path: **S-A-D-G**
Path length: **3**
Max Q length: **3**
Number of state expansions: **5**

Sequence of State Expansions: S-A-C-D-G

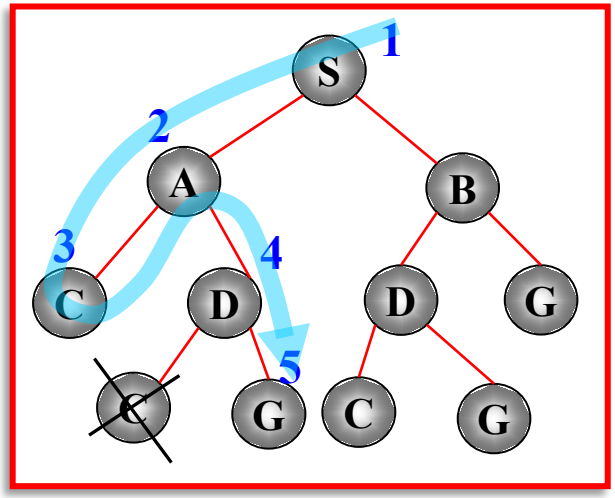
Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

Depth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS)(BS)	S,A,B
2	(AS)	(CAS)(DAS)(BS)	S,A,B,C,D
3	(CAS)	(DAS)(BS)	S,A,B,C,D
4	(DAS)	(GDAS)(BS)	S,A,B,C,D,G
5	(GDAS)	(BS)	

Search Tree



Path: S-A-D-G
Path length: 3
Max Q length: 3
Number of state expansions: 5

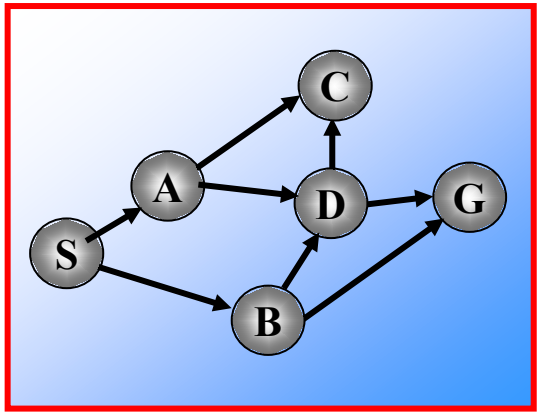
Sequence of State Expansions: S-A-C-D-G

Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

Depth-First without Visited List

Step	Q
1	(S)
2	(AS)(BS)
3	(CAS)(DAS)(BS)
4	(DAS)(BS)
5	(CDAS)(GDAS)(BS)
6	(GDAS)(BS)



Sequence of State Expansions: S-A-C-D-C-G

Path: **S-A-D-G**
Path length: **3**
Max Q length: **3**
Number of state expansions: **6**

Pick first element of Q; Add path extensions to **front** of Q.

Do NOT extend a path to a state if the resulting path would have a loop.

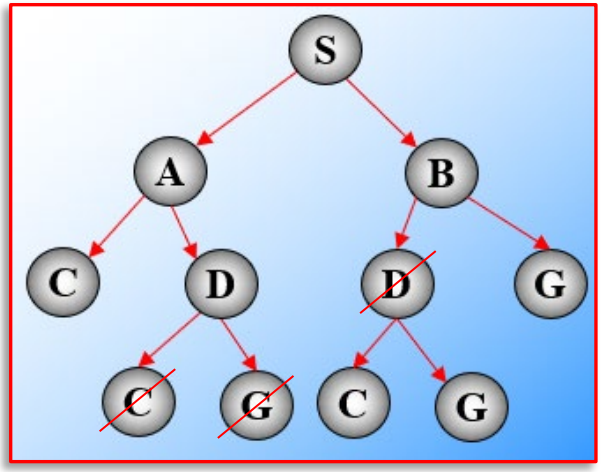
If we have visited a state along a particular path, we do not re-visit that state again in any extensions of the path

back

SEARCH

Breadth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS)(BS)	S,A,B
2	(AS)	(BS)(CAS)(DAS)	S,A,B,C,D
3	(BS)	(CAS)(DAS)(GBS)	S,A,B,C,D,G
4	(CAS)	(DAS)(GBS)	S,A,B,C,D,G
5	(DAS)	(GBS)	S,A,B,C,D,G
6	(GBS)		S,A,B,C,D,G



Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **6**

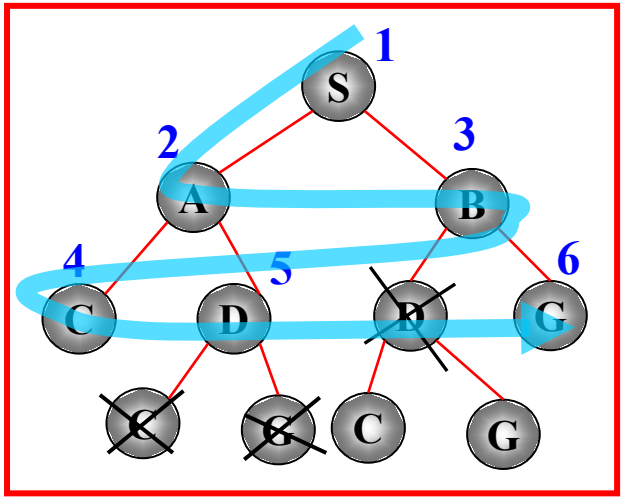
Sequence of State Expansions: S-A-B-C-D-G

Pick **first** element of Q; Add path extensions to **END** of Q.

SEARCH

Breadth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS)(BS)	S,A,B
2	(AS)	(BS)(CAS)(DAS)	S,A,B,C,D
3	(BS)	(CAS)(DAS)(GBS)	S,A,B,C,D,G
4	(CAS)	(DAS)(GBS)	S,A,B,C,D,G
5	(DAS)	(GBS)	S,A,B,C,D,G
6	(GBS)		S,A,B,C,D,G



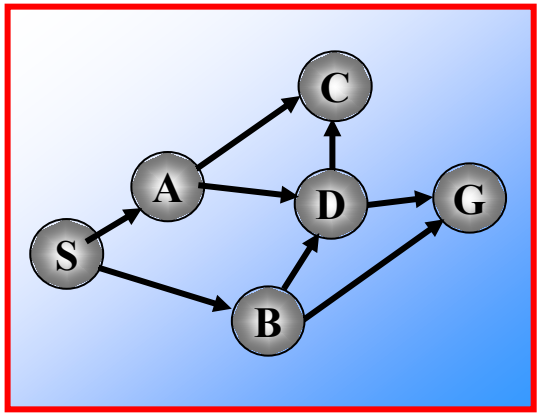
Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **6**

Sequence of State Expansions: S-A-B-C-D-G

Pick **first** element of Q; Add path extensions to **END** of Q.

Breadth-First without Visited List

Step	Q
1	(S)
2	(AS)(BS)
3	(BS)(CAS)(DAS)
4	(CAS)(DAS)(DBS)(GBS)
5	(DAS)(DBS)(GBS)
6	(DBS)(GBS)(CDAS)(GDAS)
7	(GBS)(CDAS)(GDAS)(CDBS)(GDBS)



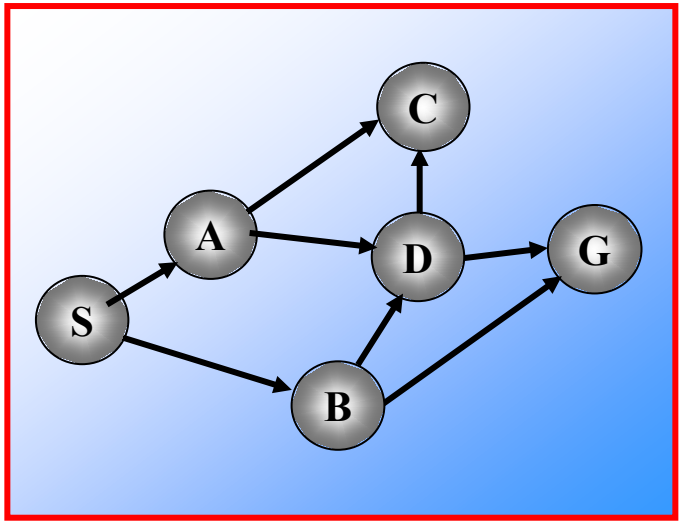
Path: **S-B-G**
Path length: **2**
Max Q length: **5**
Number of state expansions: **7**

Sequence of State Expansions: S-A-B-C-D-D-G

Pick **first** element of Q; Add path extensions to **END** of Q.

Best-First with Visited List

Step	Q	Visited
1	(10 S)	S
2	(2 AS)(3 BS)	A, B, S
3	(1 CAS)(4 DAS)(3 BS)	C, D, B, A, S
4	(4 DAS) (3 BS)	C, D, B, A, S
5	(0 GBS)(4 DAS)	G, C, D, B, A, S



Heuristic Values:
S=10, A=2, B=3, C=1, D=4, G=0

Sequence of State Expansions: S-A-C-B-G

Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **5**

For our convention in class, add the path extensions to the **front** of the Queue.

Pick **"best"** element of Q (by heuristic value); Add path extensions **anywhere** in Q.



SEARCH

Testing for the goal

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

This algorithm stops (in **Step 3**) when state (N) = G, or, in general, when state (N) satisfies the goal test.

We could have performed this test in **Step 6** as each extended path is added to **Q**. This would catch termination earlier and be perfectly correct for the searches we have covered so far.

However, performing the test in **Step 6** will be incorrect for the optimal searches. We have chosen to leave the test in **Step 3** to maintain uniformity with these future searches.

Terminology

Visited – a state M is first visited when a path to M first gets added to Q.
In general, a state is said to have been visited if it has ever shown up in a search node in Q. The intuition is that we have briefly **"visited"** them to place them on Q, but we have not yet examined them carefully.

Terminology

Visited – a state M is first visited when a path to M first gets added to Q.
In general, a state is said to have been visited if it has ever shown up in a search node in Q. The intuition is that we have briefly "visited" them to place them on Q, but we have not yet examined them carefully.

Expanded – a state M is expanded when it is the state of a search node that is pulled-off of **Q**. At that point, the descendants of M are visited and the path that led to M is extended to the eligible descendants. In principle, a state may be expanded multiple times. We sometimes refer to the search node that led to M (instead of M itself) as being expanded. However, once a node is expanded we are done with it; we will not need to expand it again. In fact, we discard it from **Q**.

Terminology

Visited – a state M is first visited when a path to M first gets added to Q. In general, a state is said to have been visited if it has ever shown up in a search node in Q. The intuition is that we have briefly "visited" them to place them on Q, but we have not yet examined them carefully.

Expanded – a state M is expanded when it is the state of a search node that is pulled-off of Q. At that point, the descendants of M are visited and the path that led to M is extended to the eligible descendants. In principle, a state may be expanded multiple times. We sometimes refer to the search node that led to M (instead of M itself) as being expanded. However, once a node is expanded we are done with it; we will not need to expand it again. In fact, we discard it from Q.

***This distinction plays a key role in our discussion of the various search algorithms; study it carefully.**

Visited States (for Any-Path search algorithms only)

PROS and CONS

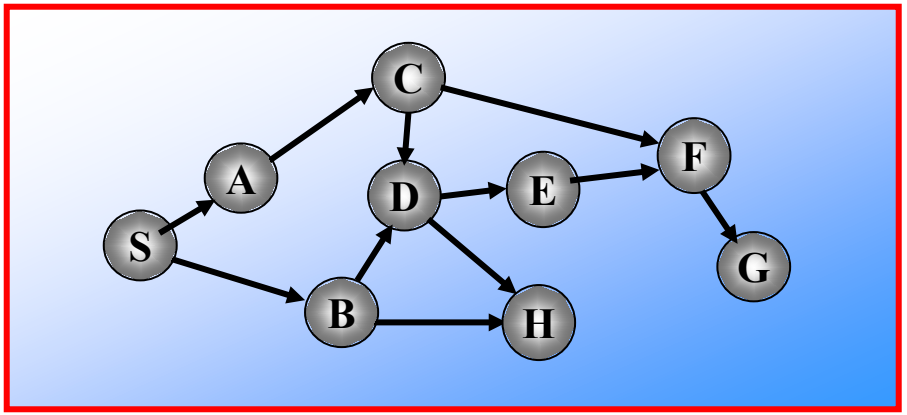
Keeping track of **visited states generally improves time efficiency** when searching graphs, without affecting correctness. Note, however, that **substantial additional space** may be required to keep track of visited states.

Why avoid revisiting states?

If all we want to do is find a path from the start to the goal, there is no advantage to adding a search node whose state is already the state of another search node.

Any state reachable from the node the second time would have been reachable from that node the first time.

e.g.



(DCAS) — place in Q? → (DBS) …

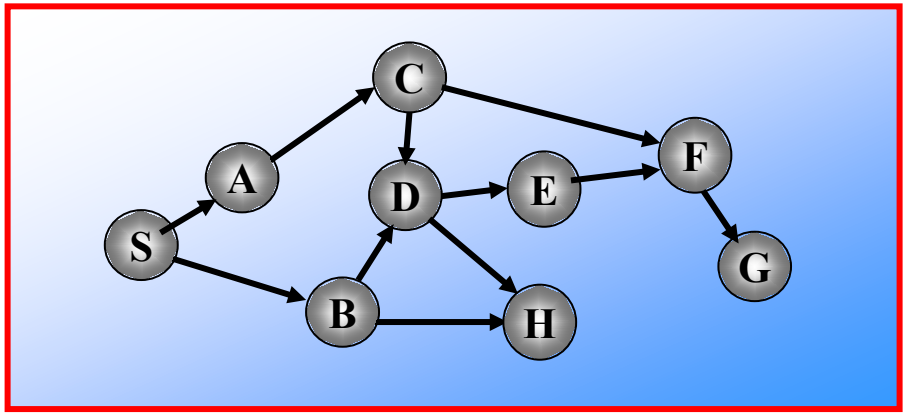
Q

Visited List S, A, B, D

back

Visited States (for Any-Path search algorithms only)

e.g.



~~(DCAS)~~ place in Q?

Q (DBS) …

Visited List S, A, B, D

Global check

Note that, when using the **Visited List**, each state will only ever have at most one path to it (search node) in **Q**.

We'll have to revisit this issue when we look at optimal searching

back

Visited States

Do we really need a Visited List to keep track of the Visited States?

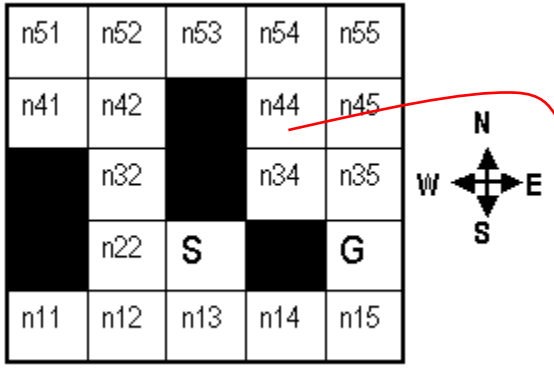
The purpose of using a **Visited list** is to identify and remember the states of all the nodes that have been placed in Q, as the algorithm runs, so as to avoid visiting them again, but how to do this efficiently?

Avoid declaring a list if possible

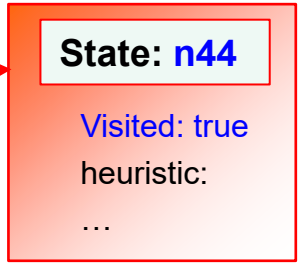
Although we speak of a **Visited list**, this is never the preferred implementation

If the **graph states** are known ahead of time as an explicit set, then space is allocated in the state itself to keep a mark; which makes both adding to Visited and checking if a state is Visited a constant time operation

e.g.



the graph states are the squares of the maze



Sample State

back

Visited States

How to speed up checking for visited states, if the states are generated on the fly?

Alternatively, as is more common in AI, if the states are generated on the fly, then a **hash table** may be used for efficient detection of previously visited states.

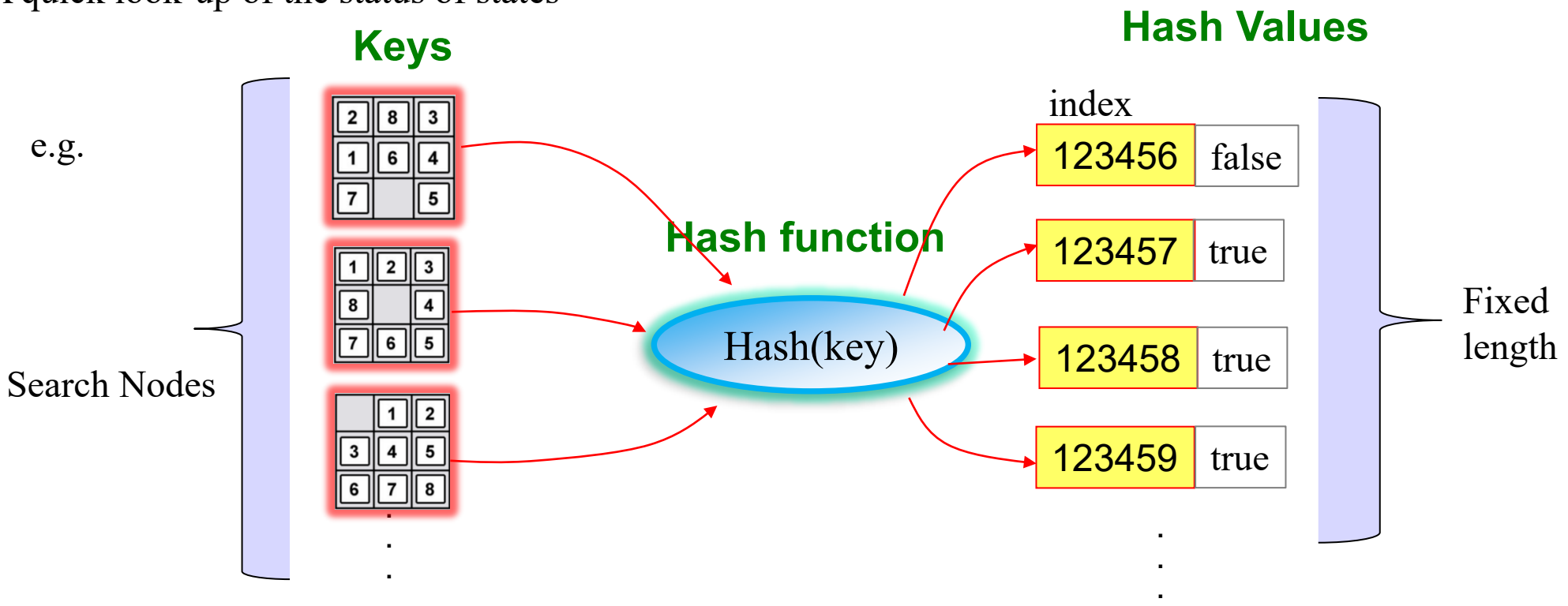
Inevitably, additional space cost is incurred

Note that, in any case, the incremental space cost of a **Visited list** will be **proportional to the number of states** – which can be very high in some problems.

SEARCH

Hash function

A quick look-up of the status of states



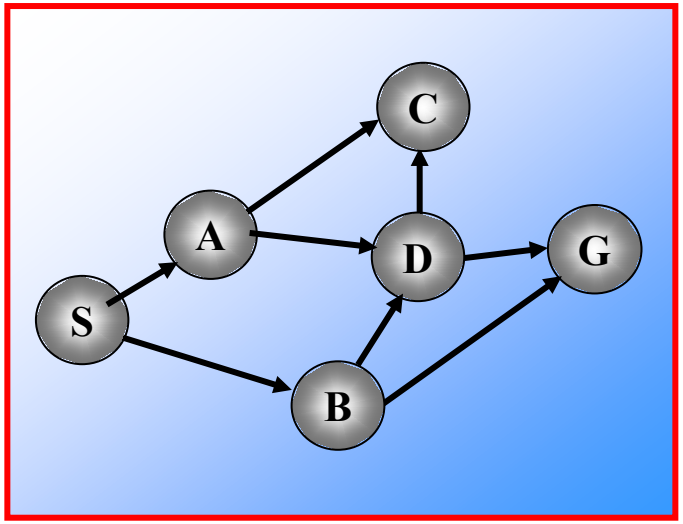
A **hash function** could be used to map large data sets of **variable length**, called **keys** to **hash values** that are of a **fixed length**. The hash values may serve as indexes to an associative array.

However, hash values may have duplicates; therefore, may cause collision between keys.

SEARCH

Best-First with Visited List

Step	Q	Visited
1	(10 S)	S
2	(2 AS)(3 BS)	A, B, S
3	(1 CAS)(4 DAS)(3 BS)	C, D, B, A, S
4	(4 DAS) (3 BS)	C, D, B, A, S
5	(0 GBS)(4 DAS)	G, C, D, B, A, S



Heuristic Values:
S=10, A=2, B=3, C=1, D=4, G=0

Sequence of State Expansions: S-A-C-B-G

Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **5**

For our convention in class, add the path extensions to the **front** of the Queue.

Pick **"best"** element of Q (by heuristic value); Add path extensions **anywhere** in Q.



Heuristic

Heuristic

- the word generally refers to a "rule of thumb", something that may be helpful in some cases but not always.
- Generally held to be in contrast to "guaranteed" or "optimal"

Heuristic function

- in search terms, a function that **computes a value for a state** (but does not depend on any path to that state) that may be helpful in guiding the search.

e.g. **Estimated distance to goal**

- this type of heuristic function depends on the state of the goal. The classic example is straight-line distance used as an estimate for actual distance in a road network. This type of information can help increase the efficiency of a search.

Implementation Issues: Finding the best node in Q

There are many possible **approaches to finding the best node in Q**.

1. **Scanning Q** to find lowest value
2. **Sorting Q** and picking the first element
3. Keeping the **Q** sorted by doing "sorted" insertions
4. Keeping **Q** as a **Priority Queue**

Which of these is best will depend among other things on how many children the nodes have on average. We will look at this in more detail later.

http://www.cplusplus.com/reference/stl/priority_queue/

Tutorial #1

Let us do some practice using the Any-Path algorithms

back